

Present Value Engine Code Walkthrough

A maintainer's guide to the present-value worksheet: field-presence solving across lump-sum, periodic and rate blocks, COLA escalation, variable-rate discounting, the life-contingency fold-in, and the underlying geometric-series mathematics.

Scope. The PV engine ([PRESVALU.pas](#)), the variable-rate ("fancy") engine and table output ([PVLXSCRN.pas](#), [pvltable.pas](#), [PVLUTIL.pas](#)), the screen layer ([PresentValueScreenUnit.pas](#)), and the shared math ([INTSUTIL.pas](#)).

Audience. Engineers maintaining or porting the financial logic. Every claim is anchored to a `file:line` reference.

Authority & location. The DOS source is authoritative. All line numbers cite the annotated mirror `legacy/src_documented/dos_source/`, which carries inline commentary; it is the same code as `legacy/src/dos_source/` but with the comments included, so its line numbers differ ([PRESVALU.pas](#) there is 1589 lines).

Actuarial. The life-contingency ("for life" / pay-on-death) integration that weaves through this engine is covered in the companion `actuarial_dos_walkthrough.pdf`; here it is noted where it hooks in.

Contents

0 How to read this document

1 Architecture & data model

1.1 The three blocks · 1.2 Records & status codes · 1.3 Rates, COLA & sentinels

2 The UI layer: five grids

2.1 Grids & containers · 2.2 The rate three-views · 2.3 COLA & Simple/Compound · 2.4 The calc trigger & the plain ↔ fancy toggle

3 Entry & classification

3.1 Enter · 3.2 FirstPass · 3.3 The field-presence → solve-path map

4 The summation mathematics

4.1 SumFormula · 4.2 Summation · 4.3 COLA & stepped escalation

5 Forward calc

6 Backward calc

7 The life-contingency fold-in

8 Variable-rate ("fancy") present value & table output

9 A worked trace & maintainer notes

10 Appendix: call graph & file:line index

0 How to read this document

The present-value screen is a worksheet that values streams of money across time. It has three editable blocks: lump-sum single payments, periodic payment streams (with optional cost-of-living escalation), and a rate /

present-value block holding the discount rate and the as-of date. As with every Per%Sense screen, the user leaves one cell blank and the engine solves for it — an amount, a date, a rate, or the total value. A "fancy" mode replaces the single discount rate with a schedule of rates over time.

Citation convention

References look like `PRESVALU.pas:1387`; the number is where the construct begins, and resolves under `legacy/src_documented/dos_source/`.

File	Role
<code>PRESVALU.pas</code>	Engine. Enter dispatcher, <code>FirstPass</code> classifier, <code>FrontwardCalc</code> (forward + rate/as-of solves), <code>BackwardCalc</code> (amount/date solves), and the <code>Summation</code> geometric-series math.
<code>PVLXSCRN.pas</code>	Variable-rate ("fancy") engine: <code>FancyFirstPass</code> , <code>FancySummation</code> , COLA escalation.
<code>PVLUTIL.pas</code>	<code>ValueOfOnePayment</code> — the single-payment discount primitive that walks a rate schedule.
<code>pvltable.pas</code>	<code>MakePVLTable</code> — merges all series by date and emits the detail + summary schedule.
<code>PresentValueScreenUnit.pas</code>	UI. Five grids, the rate three-views, COLA conversion, the calc trigger.
<code>INTSUTIL.pas</code>	Shared math (<code>exxp/lnn</code> , rate conversions, <code>YearsDif</code>) — see the amortization walk-through.

The one formula to know. The present value of a periodic stream is $\text{amtn} \times \text{Summation}$, and `Summation` is a geometric series $(1-f^n)/(1-f)$ whose ratio $f = e^{(\text{cola}-\text{rate})/P}$ nets escalation against discounting. When $\text{cola} = \text{rate}$, $f \rightarrow 1$ and the series flattens to n — the single most important edge case in the whole engine.

1 Architecture & data model

1.1 The three blocks

The worksheet is three arrays of rows plus a rate block, all discounted to a common as-of date:

```

BLOCK 1  a[]  lump-sum single payments    (date, amount, value)
BLOCK 2  b[]  periodic payment streams    (from, to, per-yr, amount, COLA, value)
BLOCK 3  c[]  rate / present-value lines   (as-of, rate, sumvalue, duration)

fancy mode adds:  cc[]  rate-line schedule (date, rate)    d  footer (xasof, simple, xvalue)
```

Each row solves the field the user left blank (the "2-of-3" rule for lumps: any two of date/amount/value give the third). `sumvalue` in BLOCK 3 is the total present value the screen reports and the backward solver targets.

1.2 Records & status codes

Per-field `inout` status codes `PETYPES.PAS:178–218`: `empty=0` (blank, the solve request), `outp=1` (computed; also the provisional `fromcalc` flag), `defp=2` (known/derived), `inp=3` (typed), `badp=-1` (flagged bad). The guard status $\geq \text{defp}$ means "known". A per-row aggregate line-status code `PETYPES.PAS:490–496` summarizes a row:

fully_specified=254 (forward-computable), contains_unknown=253 (one blank to solve), over_determined=255, missing_N=250–252.

Record	Key fields	Source
lumpsum (a)	date, amt0 (face amount at its date), val0 (PV at as-of), act0 (contingency code)	PETYPES.PAS:755
periodic (b)	fromdate/todate, peryr, amtn, cola, valn (PV of stream), ninstallments, actn	PETYPES.PAS:781
raterec	rate (stored True/continuous rate), peryr (its compounding), status (fromrate/fromapr/fromyield)	PETYPES.PAS:807
presval (c)	asof, r:raterec, sumvalue, duration	PETYPES.PAS:824
rateline (cc) / xpresval (d)	fancy rate band (date, r); fancy footer (xasof, simple, xvalue)	PETYPES.PAS:839, 851

1.3 Rates, COLA & sentinels

The stored rate is a continuously-compounded rate; cola is a continuous annual escalation rate.

df.c.colamonth selects the COLA policy: 1..12 = escalate on that month each year; CNT=98 = continuous; ANN=99 = anniversary-of-start PETYPES.PAS:151–152. Key cutoffs: teeny= 10^{-10} (zero-rate boundary), tiny= 10^{-5} (Taylor-series band). The sentinel latest.y marks a stream that runs "forever" (a perpetuity), tested as todate.y=latest.y PRESVALU.pas:531.

2 The UI layer: five grids

2.1 Grids & containers

The form hosts five grids, each backed by a global record array PresentValueScreenUnit.pas:128–135:

Grid	Container	Role
LumpSumGrid	a[]	single payments (date / amount / value)
PeriodicGrid	b[]	periodic streams (from / to / per-yr / amount / COLA / value)
PresentValueGrid	c[]	plain single-rate PV (as-of / True / Loan / Yield / value)
RatelineGrid	cc[]	variable-rate bands (date / True / Loan / Yield)
XPresValGrid	d	fancy footer (as-of / computation / value)

The grid → record marshallers (AssignLumpSumValues PresentValueScreenUnit.pas:1283, AssignPeriodicValues PresentValueScreenUnit.pas:1401) follow the universal blank-vs-typed rule: Value='' → status := empty (the field-presence signal); otherwise parse and set inp. COLA is stored as a fraction (/100) on input PresentValueScreenUnit.pas:1457.

2.2 The rate three-views

The rate block shows one rate three ways — True (effective per-period), Loan APR, and Yield — and editing any one recomputes the others. The stored r.rate is always the True (continuous) rate

PresentValueScreenUnit.pas:1543–1622:

```
// True (status fromrate): the periodic rate itself
r.rate := value/100; r.status := fromrate
Loan = YieldFromRate(r.rate, df.c.peryr)
Yield = YieldFromRate(r.rate, 1)
// Loan (status fromapr): r.rate := RateFromYield(value, df.c.peryr)
// Yield (status fromyield): r.rate := RateFromYield(value, 1)
```

When the payments-per-year changes, FixRates `PresentValueScreenUnit.pas:252` re-derives every fromapr row so the displayed APR tracks the new compounding.

2.3 COLA & Simple/Compound

COLA is entered as a percent and stored as a fraction (/100 on input, ×100 on display). The fancy footer's Computation combo sets `d.simple`: in Simple mode the three rate views all show the raw number (no compounding conversion); in Compound mode they show the converted views.

2.4 The calc trigger & the plain ↔ fancy toggle

Enter in any cell calls `DoCalculation PresentValueScreenUnit.pas:460`, which loads the used-row counts into `nlines[]`, then — in fancy mode — pins the first rate band to the epoch (`cc[1].date := earliest; datestatus := defp PresentValueScreenUnit.pas:468–469`) so the earliest payment is always covered, and calls the engine's `Enter(no_tab)`. Results flow back to the grids. `ToggleAdvanced PresentValueScreenUnit.pas:486` flips the global `pvl fancy`, swapping block geometry so BLOCK 3 becomes the rate-line block `cc`.

3 Entry & classification

3.1 Enter

`Enter(code) PRESVALU.pas:1387` runs `FirstPass`, warns before overwriting any already-determined BLOCK-3 line, re-classifies, then dispatches: frontward *and* backward → "too many unknowns"; backward → `BackwardCalc PRESVALU.pas:1525`. After a backward solve it forward-computes the remaining rate lines top-down (for `i := nlines` downto `stopat` do `FrontwardCalc(i) PRESVALU.pas:1535`; downto keeps line 1's solved result on screen). The unknown-POD path is dispatched last (if `(podunk)` then `ComputeUnknownPOD PRESVALU.pas:1550`). Actuarial entry points (`PrepareForLife`) hook in here — see the actuarial walk-through.

3.2 FirstPass

`FirstPass PRESVALU.pas:726` classifies each block (delegating to `FancyFirstPass` when `pvl fancy` `PRESVALU.pas:742`). It counts present fields to assign each row a line-status code, deletes empty rows, then resolves two flags: frontward is true only if some row is `fully_specified` and no row still contains `unknown`; backward is set inside the line-value computations when a `contains_unknown` row coexists with a fully-specified rate line. The classifier delegates to `ComputeLumpsumLineValues PRESVALU.pas:263` and `ComputePeriodicLineValues PRESVALU.pas:628` per block.

3.3 The field-presence → solve-path map

The blank field selects the path (labels match the Go port's BackwardKind). Note PV-8/PV-9 solve a BLOCK-3 cell (forward side, in FrontwardCalc); PV-1/2/4/5/6 solve a payment cell (backward side, in BackwardCalc). There is no PV-3 or PV-7.

Blank field	Block	Path	Lines
lump amount (date+value given)	1	PV-1	PRESVALU.pas:1116–1130
lump date (amount+value given)	1	PV-2	PRESVALU.pas:1149–1175
periodic amount (both dates+value)	2	PV-4	PRESVALU.pas:1192–1206
periodic to-date (from+amt+value)	2	PV-5	PRESVALU.pas:1216–1254
periodic from-date (to+amt+value)	2	PV-6	PRESVALU.pas:1255–1338
BLOCK-3 rate (payments + sumvalue)	3	PV-8	PRESVALU.pas:902–945
BLOCK-3 as-of date	3	PV-9	PRESVALU.pas:972–1027

4 The summation mathematics

The PV of one periodic stream is $\text{amtn} \times \text{Summation}$. Throughout: r = discount rate, c = COLA, P = RealPerYr(peryr) (real payments/yr), n = ninstallments, and YearsDif(asof,date) is negative for future payments (so the exponential discounts).

4.1 SumFormula — the geometric kernel

SumFormula(lnf, n) PRESVALU.pas:362 is the closed form of $\sum_{i=0}^{n-1} e^{i \cdot \text{lnf}}$:

$$\text{SumFormula} = (1 - e^{(n \cdot \text{lnf})}) / (1 - e^{(\text{lnf})}), \quad \rightarrow n \quad \text{when } |\text{lnf}| < \text{teeny}$$

The teeny cutoff (per-period growth ≈ 0) degenerates the series to n equal terms PRESVALU.pas:366–367. A tiny-band branch swaps in 2nd-order Taylor expansions of $1-e^x$ so the 0/0-shaped ratio stays accurate near zero.

4.2 Summation

Summation(k,j) PRESVALU.pas:517 sets the per-period net log-growth $\text{lnf} := (\text{cola} - r)/P$ — escalation working against discounting — and (in the common closed-form path) returns:

$$\text{Summation} = e^{(r \cdot \text{since})} \cdot (1 - f^n)/(1 - f) \cdot [\text{phase factor}], \quad f = e^{\text{lnf}} = e^{((c-r)/P)}$$

where since measures time from the as-of date to the stream's reference point PRESVALU.pas:564–614. The branch picks whether to measure from a "standard loan date" one period before the first payment (when asof $\leq$ fromdate; since_from := true, then AddPeriod(stdloandate,...,subtract)) or from the to-date, with a phase factor $\text{ff} := \text{exxp}(-r \cdot \text{rate}/\text{RealPerYr(peryr)})$ correcting the one-period offset.

The cola = rate degeneracy. When $c = r$, $\text{lnf} = 0$, $f = 1$, and the geometric ratio $(1-f^n)/(1-f)$ is 0/0. The teeny branch handles it by collapsing to n equal terms: every escalated payment discounts back to exactly the same present value, so the stream is worth n times one payment's PV. Every backward solver special-cases this — PV-5/PV-6 use AddNPeriods(..., round(val/amtn)) PRESVALU.pas:1224 to dodge the division by zero.

Two other paths exist: a perpetuity guard (if COLA $\geq$ rate and the stream runs forever, `todate.y = latest.y`, the value is infinite — flagged `badp`, [PRESVALU.pas:531–534](#)) and an exact period-by-period sum used for weekly/biweekly, life-contingent, or explicitly-exact modes [PRESVALU.pas:541–560](#).

4.3 COLA & stepped escalation

When the COLA escalates in steps (anniversary or a chosen calendar month, not continuously), `SummationForSteppedCola(k, j)` [PRESVALU.pas:399](#) applies the multiplier once per year. It walks year by year: within each anniversary year the payment amount is constant and summed by a closed-form `SumFormula` over the in-year installments; at each `coladate` crossing the running `current_pmt` is multiplied by `exxp(nfullyears·cola)` [PRESVALU.pas:480, 487](#). `coladate` is seeded from `fromdate` — bumped to the chosen `colamonth` when `1..12`, otherwise to the next anniversary [PRESVALU.pas:414–419](#).

5 Forward calc

`FrontwardCalc(k)` [PRESVALU.pas:862](#) is the forward evaluator and the home of the rate / as-of solves. The unknown-value path sums $\sum \text{amt0} \cdot e^{r \cdot \Delta} + \sum \text{amtn} \cdot \text{Summation}(k, j)$ over all rows ([PRESVALU.pas:880–896](#)), folding in life-contingency weights and the POD where active.

PV-8 (unknown rate). A damped secant iteration: `r.rate := 0.1` first guess, the step `diff := (sumvalue-sum)·diff/denom` (numeric derivative across two evaluations) clamped to ± 0.04 , repeating until $|\text{diff}| < \text{teeny}$ or 30 iterations [PRESVALU.pas:904–945](#).

PV-9 (unknown as-of date). A near-closed-form fixed point seeded at `asof.y := 100` (year 2000), iterating `diff := ln(sumvalue/sum)/r.rate` until $|\text{diff}| < 0.002$ years ($\approx < 1$ day) or 10 passes [PRESVALU.pas:972–1027](#) — the source comment notes it "doesn't really require iteration".

6 Backward calc

`BackwardCalc` [PRESVALU.pas:1071](#) solves the one blank payment field. It first derives the row's target value by subtracting every known row's value (and, under life, the POD) from the screen `sumvalue` [PRESVALU.pas:1084–1092](#), then routes by which field is blank:

Path	Solve	Method	Line
PV-1	lump amount	algebraic inverse: <code>amt0 := val0·exxp(r.rate·YearsDif(date, asof))</code>	PRESVALU.pas:1117
PV-2	lump date	Newton on <code>amt0·exxp(r·yrs)=val0</code> ; step <code>diff:=(val0-val)/dval</code> , clamped, aborts past <code>y>199</code>	PRESVALU.pas:1151
PV-4	periodic amount	direct: <code>amtn := valn/Summation(1, j)</code>	PRESVALU.pas:1201
PV-5	periodic to-date	analytic <code>AddYears</code> approximation, then ± 1 -period refinement (or <code>AddNPeriods</code> when <code>cola=rate</code>)	PRESVALU.pas:1216
PV-6	periodic from-date	two-pass analytic approximation, then the same ± 1 -period snap	PRESVALU.pas:1255

The date solves invert the same `Summation` they forward-evaluate: they compute a continuous-time first approximation, count installments with `NumberOfInstallments(..., ON_OR_BEFORE)`, then nudge the date one

period each way and keep whichever $\text{altval} = \text{amtn} \cdot \text{Summation}(1, j)$ is closest to the target `PRESVALU.pas:1227–1245, 1299–1317`.

7 The life-contingency fold-in

The `{$ifdef ACTU}` build weights payments by survival probability and adds a Payment-on-Death term. The hooks (active when `fold_in_life`):

- **Lump rows.** `if (fold_in_life) then val0:=val0*LifeProb(date,a[i]^act0) in`
`ComputeLumpsumLineValues` `PRESVALU.pas:313`, and again in the forward sum `PRESVALU.pas:884`.
- **Periodic rows.** The contingent case forces the exact period-by-period walk and multiplies each payment by survival: `if (fold_in_life) then part:=part*LifeProb(t,b[j]^actn)` `PRESVALU.pas:549` (and the stepped variant at `PRESVALU.pas:427`).
- **Payment-on-Death.** `if (fold_in_life) then sumvalue:=sumvalue+PodValue(asof,r.rate)` folds the death benefit into the total `PRESVALU.pas:896` (also at `PRESVALU.pas:921, 1001`). The backward target subtracts it first `PRESVALU.pas:1085–1092`. When POD itself is blank, `ComputeUnknownPOD` back-solves it from the target `PRESVALU.pas:1550`.

The two-life cases also divide the recovered face amount by `LifeProb` on the lump-amount inverse `PRESVALU.pas:1119–1124`, with a "beyond life span" guard. The life tables and contingency codes are documented in `actuarial_dos_walkthrough.pdf`.

8 Variable-rate ("fancy") present value & table output

In fancy mode, `FancyFirstPass` `PVLXSCRN.pas:46` classifies the rows and `FancySummation(j)` `PVLXSCRN.pas:451` values one periodic series by an exact period-by-period walk, calling `ValueOfOnePayment` `PVLUTIL.pas:69` per payment — the kernel that discounts a single payment by walking the rate schedule band by band. COLA is applied at emission via `UpdateAmountWithCola` `PVLXSCRN.pas:152`.

Dead "fast" paths. The intended closed-form "fast" branch in `FancySummation` was never written — the source notes it is an unimplemented stub, so the exact per-payment walk is what actually runs `PVLXSCRN.pas:445–450`. `cc[1]` is force-anchored to earliest and is not user-editable.

`MakePVLTable` `pvltable.pas:152` merges every lump and periodic series into one date-ordered stream and prints detail + cumulative rows. `SortPayments` `pvltable.pas:283` sorts and seeds each series' cursor (capping perpetual streams). `SelectNextPayment` `pvltable.pas:356` is the merge heart: it finds the earliest pending date across all series and sums any co-dated payments. `TimeIsRipe` `pvltable.pas:424` decides when a cumulative subtotal is due. `PrintNextPayment` `pvltable.pas:619` discounts the merged payment — `ValueOfOnePayment(q,t)` (fancy) or $q \cdot e^{-r \cdot \text{YearsDif}(t, \text{asof})}$ (plain) — and accumulates the running total.

9 A worked trace & maintainer notes

Trace. One periodic stream: \$1,000/month for 10 years, 5% rate, no COLA, value blank (solve the value).

#	What happens	Code
1	Cells typed; value blank → <code>valnstatus := empty</code> . Rate stored as the True rate.	<code>PresentValueScreenUnit.pas:1401</code>
2	Enter → <code>DoCalculation</code> → <code>Enter(no_tab)</code> .	<code>PresentValueScreenUnit.pas:460</code>
3	<code>FirstPass: ninstallments = 120</code> ; row <code>fully_specified</code> (amount present, value blank) → <code>frontward</code> .	<code>PRESVALU.pas:726</code>
4	<code>FrontwardCalc</code> unknown-value path: <code>valn = 1000·Summation</code> ; <code>lnf=(0-0.05)/12</code> , <code>f<1</code> , so the geometric sum is finite.	<code>PRESVALU.pas:892</code>
5	<code>valnstatus := outp</code> ; <code>sumvalue</code> aggregates; results written back, value coloured as computed.	<code>PRESVALU.pas:880</code>

Now blank the amount and supply a value: step 4 becomes PV-4, `amtn = valn/Summation(1,j)` `PRESVALU.pas:1201`. Set `cola = 0.05` equal to the rate: `lnf = 0`, the summation flattens to `n = 120`, and the stream is worth exactly 120 discounted payments — the degeneracy of \$4.2.

Maintainer notes & known quirks

- **`cola = rate` is the load-bearing edge case.** The geometric ratio is 0/0; the teeny branch collapses it to `n`. Every forward and backward path special-cases it. Test it explicitly.
- **Rate stored as True (continuous).** Three views are derived; editing Loan/Yield round-trips through `RateFromYield/YieldFromRate`. A `peryr` change re-derives `fromapr` rows via `FixRates` `PresentValueScreenUnit.pas:252`.
- **Perpetuities** (`today.y = latest.y`) are finite only when `rate > cola`; otherwise the engine flags an infinite value `PRESVALU.pas:531`.
- **Fancy "fast" paths are dead code** `PVLXSCRN.pas:445`; only the exact installment summation runs.
- **The `{ifdef V_3} const_signal` block in PV-6** `PRESVALU.pas:1261–1265` is dead in the authoritative build (`V_3` never defined) and is intentionally not ported.
- **Two-pass `FirstPass` and the `downto` loop in `Enter` are deliberate:** the second pass re-derives state cleared by the overwrite warning, and `downto` preserves line 1's backward result `PRESVALU.pas:1535`.

10 Appendix: call graph & file:line index

```

Enter(code)                                PRESVALU.pas:1387
├─ FirstPass (classify; set frontward/backward) PRESVALU.pas:726
│   ├── ComputeLumpsumLineValues / ComputePeriodicLineValues :263 / :628
│   └─ (fancy) FancyFirstPass                PVLXSCRN.pas:46
├─ (backward) BackwardCalc                    PRESVALU.pas:1071
│   ├── derive target (subtract known val0/valn - POD)
│   ├── PV-1 amount = val·exp(r·(date-asof))           :1117
│   ├── PV-2 date   Newton                           :1151
│   ├── PV-4 amount = valn/Summation(1,j)               :1201
│   ├── PV-5 to-date / PV-6 from-date (approx + ±1-period snap) :1216 / :1255
│   └─ (frontward) FrontwardCalc(i) downto stopat      PRESVALU.pas:862
│       ├── unknown value  $\Sigma \text{amt0} \cdot \exp(r \cdot \Delta) + \Sigma \text{amtn} \cdot \text{Summation}$  :880
│       ├── PV-8 rate secant (numeric derivative)       :904
│       ├── PV-9 asof fixed-point  $\Delta = \ln(SV/\text{sum})/r$  :972
│       └─ Summation  $\rightarrow$  SummationForSteppedCola  $\rightarrow$  SumFormula :517 / :399 / :362

```

Variable-rate: FancySummation $\rightarrow$ ValueOfOnePayment (per band) PVLXSCRN:451 / PVLUTIL:69
 Table: MakePVLTable $\rightarrow$ SelectNextPayment / PrintNextPayment pvltable:152 / 356 / 619
 UI: DoCalculation $\rightarrow$ Enter(no_tab) PresentValueScreenUnit.pas:460
 Life: LifeProb / PodValue fold-in PRESVALU.pas:313 / 896

Symbol	Reference
Records (lump / periodic / rate / presval)	PETYPES.PAS:755 / 781 / 807 / 824
Status & line-status codes	PETYPES.PAS:178–218 / 490–496
Enter / FirstPass	PRESVALU.pas:1387 / 726
FrontwardCalc / BackwardCalc	PRESVALU.pas:862 / 1071
Summation / SumFormula / stepped COLA	PRESVALU.pas:517 / 362 / 399
LifeProb / PodValue fold-in	PRESVALU.pas:313 / 896
FancySummation / FancyFirstPass	PVLXSCRN.pas:451 / 46
ValueOfOnePayment	PVLUTIL.pas:69
MakePVLTable / SelectNextPayment / PrintNextPayment	pvltable.pas:152 / 356 / 619
UI grids / calc trigger / FixRates	PresentValueScreenUnit.pas:128 / 460 / 252

Per%Sense DOS $\rightarrow$ Go port. Line numbers cite `legacy/src_documented/dos_source/`, the annotated mirror with inline commentary. The life-contingency integration is documented in `actuarial_dos_walkthrough.pdf`. Companions: `amortization_dos_walkthrough.pdf`, `mortgage_dos_walkthrough.pdf`.